SCO 310 / SIT 300 — Component Programming Project Report

QR Component Studio

A QR Code generator built from reusable JavaBeans components

Submission detail	Information
Student name	`[STUDENT NAME]`
Registration number	`[REGISTRATION NUMBER]`
Lecturer	Dr. Bernard Ondara, Ph.D.
Institution	Kenyatta University — Department of Computing
Unit	SCO 310 / SIT 300 — Component Programming
Submission date	`[DATE]`

Table of contents

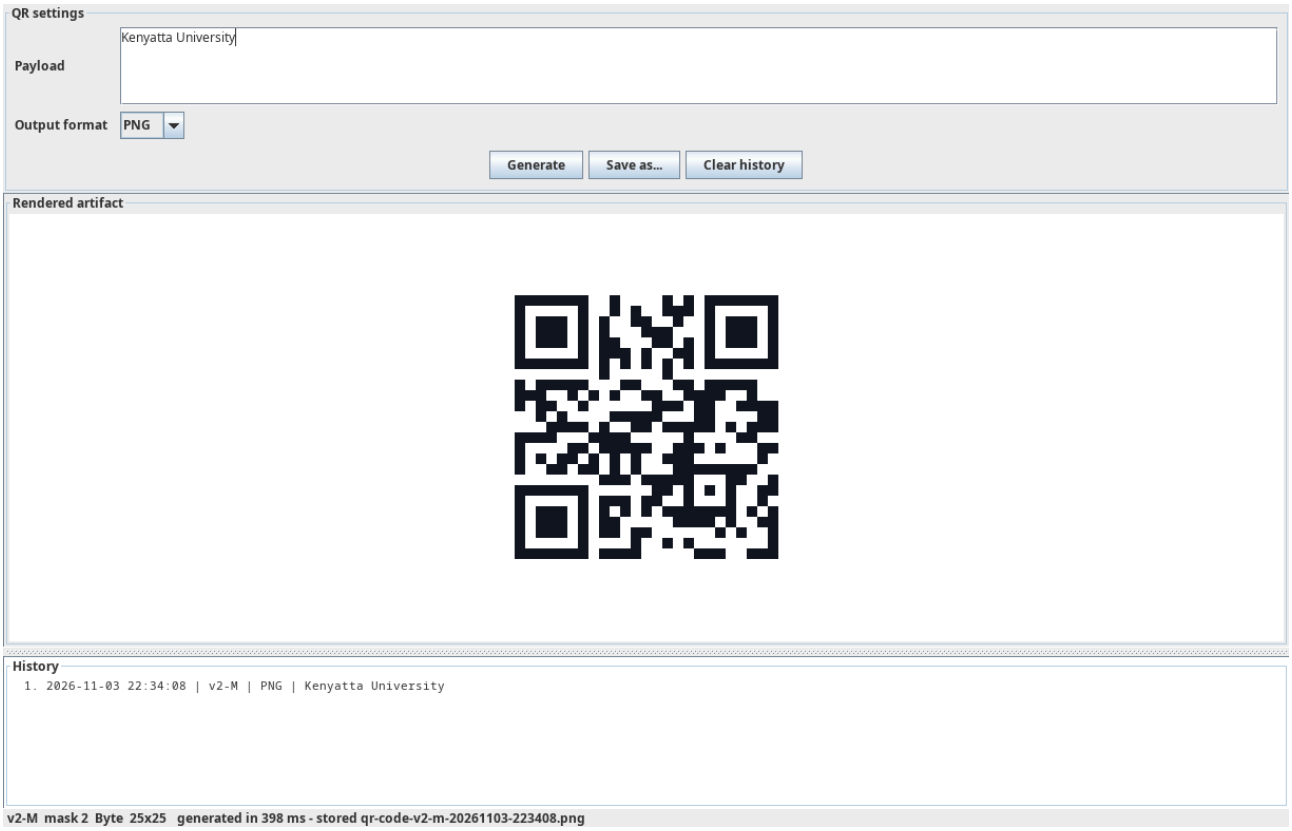
1. The application
2. Output formats
3. Main program logic
4. Components used
5. QR Code generation
6. Configuration and saved data
7. Testing
8. Problems fixed

9. Conclusion
10. References

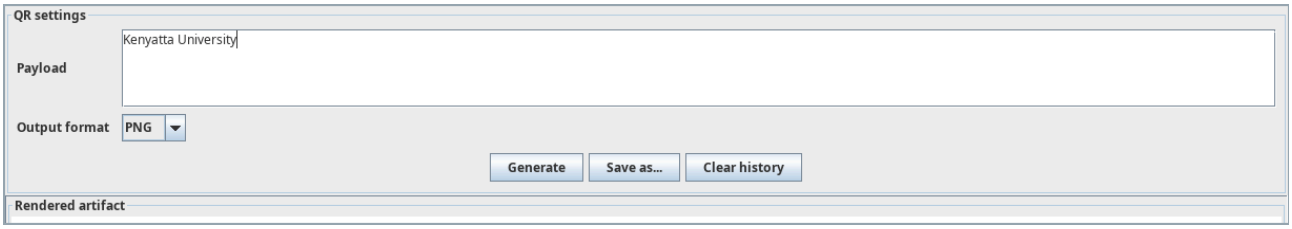
---

1. The application

QR Component Studio accepts text, creates a QR Code, displays it and saves the generated file. The same window also shows the selected output format, recent history and details about the generated symbol.



1.1 Entering the text and choosing the format



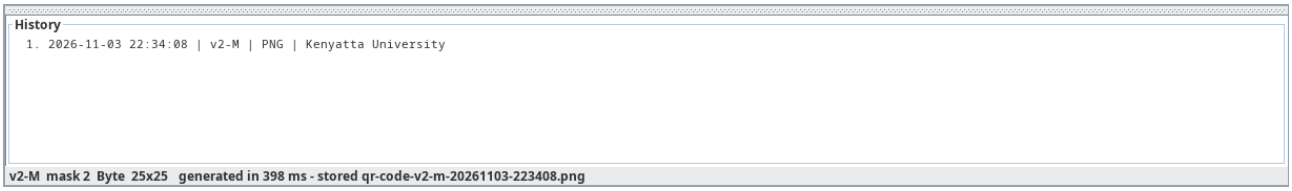
The payload box contains the text to be encoded. The output selector chooses the renderer used for the result. When **Generate** is clicked, the text is passed to

`QrStudioFacadeBean`, which starts the generation process.

1.2 Result, history and saved file



The encoder creates the QR matrix and the selected renderer turns it into the requested output. The result is displayed in the preview area.



After generation, the history area records the operation. The status line shows the QR version, error-correction level, mask, generation time and saved filename.

---

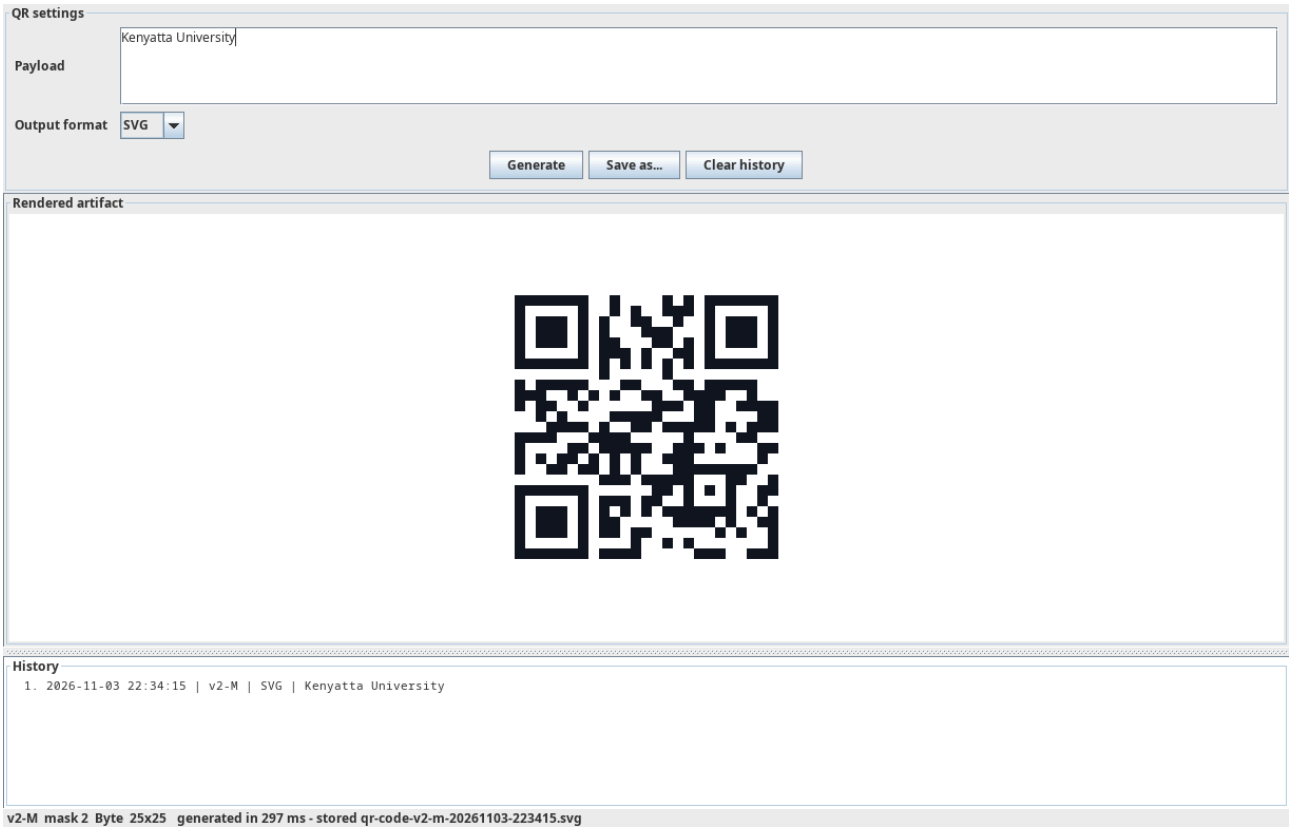
2. Output formats

The application uses separate renderer components. Changing the format does not change the encoder, history or file-storage components.

2.1 PNG

The PNG result is shown in Section 1. It is produced by `PngRendererBean` using the configured module size, quiet zone and colours.

2.2 SVG



`SvgRendererBean` creates vector output from the same QR matrix. The image remains clear when it is resized.

2.3 Text



`AsciiRendererBean` changes the dark and light modules into text characters. The application shows the result in a scrollable text area.

2.4 Other available formats

The project also contains `RoundedModuleRendererBean` for rounded PNG output and `PdfRendererBean` for vector PDF output. Both implement the same `QrCodeRenderer` interface.

---

3. Main program logic

3.1 Encoder interface

Encoder interface

```java
package ke.ku.sco310.qr.api;

/** Provided interface of an encoder component. */
public interface QrCodeEncoder extends java.io.Serializable {

    /** Encodes content into a QR Code symbol. */
    QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException;

    /** @return capability metadata of this encoder implementation. */
    EncoderInfo getEncoderInfo();

    /** @return stable identifier of the component instance, used in logs and saved results. */
    String getComponentId();
}
```

`QrCodeEncoder` defines the method used to encode text. The built-in encoder and the optional ZXing encoder both implement this interface.

3.2 Encoder component

Encoder component and its properties

```java
package ke.ku.sco310.qr.components;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.engine.QrEngine;
import ke.ku.sco310.qr.engine.QrTables;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.beans.VetoableChangeListener;
import java.beans.VetoableChangeSupport;
import java.io.Serializable;
import java.util.Arrays;

/** In-house encoder component: a JavaBean wrapping the pure-Java engine of this project. */
public class PureJavaQrEncoderBean implements QrCodeEncoder, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier (also recorded inside every symbol this bean produces). */
    public static final String COMPONENT_ID = "encoder.pure-java.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private transient VetoableChangeSupport vetoableChanges = new VetoableChangeSupport(this);

    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean boostErrorCorrection = false;
    private int minimumVersion = 1;
    private int maximumVersion = 40;
    private int forcedMaskPattern = -1;

    /** Mandatory JavaBeans no-argument constructor. */
    public PureJavaQrEncoderBean() {
    }

    // ------------------------------------------------------------------ provided interface

    @Override
    public QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException {
        QrErrorCorrection effective = level == null ? defaultErrorCorrection : level;
        return QrEngine.encode(content, effective, boostErrorCorrection, minimumVersion,
                maximumVersion, forcedMaskPattern, COMPONENT_ID);
    }

    /** Convenience overload used by the desktop application: uses the default level property. */
    public QrCode encode(String content) throws QrEncodingException {
        return encode(content, defaultErrorCorrection);
    }

    @Override
    public EncoderInfo getEncoderInfo() {
        EncoderInfo info = new EncoderInfo();
        info.setComponentId(COMPONENT_ID);
        info.setProviderName("SCO310 in-house QR engine");
        info.setProviderVersion("1.0.0");
        info.setProviderLicence("Student project - course work");
        info.setEngineDescription("Pure-Java ISO/IEC 18004 encoder: numeric, alphanumeric, byte and "
                + "kanji modes with a mixed-mode dynamic-programming segmenter, UTF-8 ECI designators, "
                + "Reed-Solomon over GF(2^8)/0x11D, all eight mask patterns scored with the four-part "
                + "penalty function, versions 1-40.");
        info.setMinimumVersion(1);
        info.setMaximumVersion(40);
        info.setMaximumByteCapacity(QrTables.maximumByteCapacity(QrErrorCorrection.LOW));
        info.setThirdParty(false);
        info.setRequiresOptionalLibrary(false);
        info.setSupportedModes(Arrays.asList(QrEncodingMode.NUMERIC, QrEncodingMode.ALPHANUMERIC,
                QrEncodingMode.BYTE, QrEncodingMode.KANJI, QrEncodingMode.ECI));
        return info;
    }

    @Override
    public String getComponentId() {
        return COMPONENT_ID;
    }
```

`PureJavaQrEncoderBean` stores the encoder settings and passes the work to the QR engine. Its main properties include the default error-correction level, minimum version, maximum version and forced mask.

3.3 Error correction

Reed-Solomon error correction

```java
package ke.ku.sco310.qr.engine;

/** Reed-Solomon error-correction codeword generator for QR Code symbols. */
final class ReedSolomon {

    private ReedSolomon() {
    }

    /**
     * @param data   data codewords
     * @param degree number of error-correction codewords to generate
     * @return the error-correction codewords (the remainder polynomial)
     */
    static byte[] computeRemainder(byte[] data, int degree) {
        if (degree < 0) {
            throw new IllegalArgumentException("Negative degree: " + degree);
        }
        int[] divisor = computeDivisor(degree);
        int[] remainder = new int[degree];
        for (byte b : data) {
            int factor = (b & 0xFF) ^ remainder[0];
            System.arraycopy(remainder, 1, remainder, 0, degree - 1);
            remainder[degree - 1] = 0;
            for (int i = 0; i < degree; i++) {
                remainder[i] ^= GaloisField.multiply(divisor[i], factor);
            }
        }
        byte[] result = new byte[degree];
        for (int i = 0; i < degree; i++) {
            result[i] = (byte) remainder[i];
        }
        return result;
    }

    /**
     * @param degree degree of the wanted divisor polynomial
     * @return coefficients of the divisor, highest order term omitted (it is always 1)
     */
    static int[] computeDivisor(int degree) {
        if (degree < 1 || degree > 255) {
            throw new IllegalArgumentException("Invalid divisor degree: " + degree);
        }
        int[] result = new int[degree];
        result[degree - 1] = 1; // start with the monomial x^0
        int root = 1;
        for (int i = 0; i < degree; i++) {
            for (int j = 0; j < result.length; j++) {
                result[j] = GaloisField.multiply(result[j], root);
                if (j + 1 < result.length) {
                    result[j] ^= result[j + 1];
                }
            }
            root = GaloisField.multiply(root, 0x02);
        }
        return result;
    }

    /** @return the divisor polynomial of {@code degree}, as readable hexadecimal coefficients. */
    static String describeDivisor(int degree) {
        StringBuilder sb = new StringBuilder("x^").append(degree);
        int[] divisor = computeDivisor(degree);
        for (int i = 0; i < divisor.length; i++) {
            sb.append(" + ").append(Integer.toHexString(divisor[i]).toUpperCase(java.util.Locale.ROOT))
              .append("x^").append(degree - 1 - i);
        }
        return sb.toString();
    }
}
```

`ReedSolomon` creates the error-correction codewords. These codewords allow a damaged QR Code to remain readable within the limit of its selected error-correction level.

3.4 PNG creation

PNG renderer

```java
package ke.ku.sco310.qr.render;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrRenderingException;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.editor.HexColorPropertyEditor;

import java.awt.Color;
import java.awt.Graphics2D;
import java.awt.image.BufferedImage;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.Serializable;
import java.util.Locale;
import javax.imageio.ImageIO;

/** Raster renderer component: draws the symbol as a square PNG image. */
public class PngRendererBean implements QrCodeRenderer, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "renderer.png.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);

    private int moduleSize = 8;
    private int quietZone = 4;
    private Color foregroundColor = Color.BLACK;
    private Color backgroundColor = Color.WHITE;

    public PngRendererBean() {
    }

    @Override
    public RenderedArtifact render(QrCode symbol) throws QrRenderingException {
        BufferedImage image = renderToImage(symbol);
        try {
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            if (!ImageIO.write(image, "png", out)) {
                throw new QrRenderingException(COMPONENT_ID, "No PNG writer is registered in this JVM");
            }
            RenderedArtifact artifact = new RenderedArtifact();
            artifact.setContent(out.toByteArray());
            artifact.setFormatName("PNG");
            artifact.setFileExtension("png");
            artifact.setMimeType("image/png");
            artifact.setWidth(image.getWidth());
            artifact.setHeight(image.getHeight());
            artifact.setTextual(false);
            artifact.setRendererId(COMPONENT_ID);
            return artifact;
        } catch (IOException e) {
            throw new QrRenderingException(COMPONENT_ID, "Could not encode the PNG image", e);
        }
    }

    /** Extra provided method used by the Swing assembly (and by the automated tests) so that the in-memory image is available without a round trip through... */
    public BufferedImage renderToImage(QrCode symbol) {
        int pixelSize = getPixelSize(symbol);
        BufferedImage image = new BufferedImage(pixelSize, pixelSize,
                backgroundColor.getAlpha() == 255 && foregroundColor.getAlpha() == 255
                        ? BufferedImage.TYPE_INT_RGB : BufferedImage.TYPE_INT_ARGB);
        Graphics2D g = image.createGraphics();
        try {
            g.setColor(backgroundColor);
```

`PngRendererBean` creates the image, paints the background and draws every dark QR module. The final image is returned as a `RenderedArtifact`.

3.5 Generation process

Generation process in the facade

```java
package ke.ku.sco310.qr.facade;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrCodeRepository;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.api.QrGenerationResult;
import ke.ku.sco310.qr.api.QrHistoryEntry;
import ke.ku.sco310.qr.api.QrRenderingException;
import ke.ku.sco310.qr.api.QrStorageException;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.event.QrCodeGeneratedEvent;
import ke.ku.sco310.qr.event.QrCodeGenerationListener;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.File;
import java.io.Serializable;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.CopyOnWriteArrayList;

/** Composite (facade) component that assembles the encoder, the renderer, the repository, the history and the listener components into the process "ge... */
public class QrStudioFacadeBean implements Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "facade.qr-studio.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private final CopyOnWriteArrayList<QrCodeGenerationListener> listeners = new CopyOnWriteArrayList<>();

    // --- required interfaces (context dependencies) ------------------------------
    private QrCodeEncoder encoder;
    private QrCodeRenderer renderer;
    private QrCodeRepository repository;
    private QrHistoryBean history;

    // --- configuration properties ------------------------------------------------
    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean autoSaveArtifacts = true;
    private boolean autoSaveHistory = true;
    private String fileBaseName = "qr-code";
    private int generatedCount;

    public QrStudioFacadeBean() {
    }

    public String getComponentId() {
        return COMPONENT_ID;
    }

    // ---------------------------------------------------------------------- provided interface

    /** Encodes, renders and (when configured) stores one payload using the injected components. */
    public QrGenerationResult generate(String content, QrErrorCorrection level)
            throws QrEncodingException, QrRenderingException, QrStorageException {
        return generate(content, level, null);
    }

    /** Variant of #generate(String, that temporarily uses a caller supplied renderer component instead of the injected one. */
    public QrGenerationResult generate(String content, QrErrorCorrection level,
                                       QrCodeRenderer rendererOverride)
            throws QrEncodingException, QrRenderingException, QrStorageException {
        requireWired();
        long start = System.nanoTime();
        QrCodeRenderer effectiveRenderer = rendererOverride == null ? renderer : rendererOverride;

        QrCode symbol = encoder.encode(content, level == null ? defaultErrorCorrection : level);
        RenderedArtifact artifact = effectiveRenderer.render(symbol);

        File stored = null;
```

`QrStudioFacadeBean` controls the main process:

1. encode the text;
2. render the QR Code;
3. save the generated file;
4. update the history; and
5. notify registered listeners.

3.6 Loading and connecting components

Loading and connecting components

```
package ke.ku.sco310.qr.facade;

import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCodeDecoder;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrMicroEncoder;
import ke.ku.sco310.qr.components.OptionalComponentSupport;
import ke.ku.sco310.qr.log.AuditLogListenerBean;
import ke.ku.sco310.qr.store.FileSystemRepositoryBean;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.beans.BeanInfo;
import java.beans.IntrospectionException;
import java.beans.Introspector;
import java.beans.PropertyDescriptor;
import java.io.File;
import java.io.Serializable;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.ServiceConfigurationError;
import java.util.ServiceLoader;

/** Component broker and assembler. */
public class ComponentRegistry {

    private final List<String> configurationNotes = new ArrayList<>();

    /** @return notes about profile properties that the selected implementations do not declare. */
    public List<String> getConfigurationNotes() {
        return new ArrayList<>(configurationNotes);
    }

    /** Role names used by the profile and by the components table. */
    public static final String ROLE_ENCODER = "encoder";
    public static final String ROLE_MICRO_ENCODER = "micro-encoder";
    public static final String ROLE_RENDERER = "renderer";
    public static final String ROLE_DECODER = "decoder";
    public static final String ROLE_REPOSITORY = "repository";
    public static final String ROLE_HISTORY = "history";
    public static final String ROLE_LISTENER = "listener";
    public static final String ROLE_FACADE = "facade";

    /**
     * @return every known component implementation, in a stable order, with capability metadata.
     */
    public List<ComponentDescriptor> listComponents() {
        List<ComponentDescriptor> catalogue = new ArrayList<>();
        for (QrCodeEncoder provider : providers(QrCodeEncoder.class)) {
            ComponentDescriptor descriptor = descriptor(ROLE_ENCODER, provider.getComponentId(), provider);
            if (provider.getEncoderInfo().isRequiresOptionalLibrary()) {
                descriptor.requires("optional third-party encoder library");
            }
            if (!OptionalComponentSupport.isZxingAvailable()
                    && provider.getEncoderInfo().isRequiresOptionalLibrary()) {
                descriptor.unavailable("com.google.zxing.core is not on the class path");
            }
            catalogue.add(descriptor);
        }
        for (QrMicroEncoder provider : providers(QrMicroEncoder.class)) {
            catalogue.add(descriptor(ROLE_MICRO_ENCODER, provider.getComponentId(), provider));
        }
        for (QrCodeRenderer provider : providers(QrCodeRenderer.class)) {
            catalogue.add(descriptor(ROLE_RENDERER, provider.getRendererId(), provider));
        }
        for (QrCodeDecoder provider : providers(QrCodeDecoder.class)) {
```

`ComponentRegistry` loads the selected components, applies their property values and checks that the required connections are available.

3.7 History

History component

```java
package ke.ku.sco310.qr.store;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrHistoryEntry;

import java.beans.IndexedPropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.beans.VetoableChangeListener;
import java.beans.VetoableChangeSupport;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** In-memory history component whose main property is an indexed property. */
public class QrHistoryBean implements Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "history.bounded-memory.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private transient VetoableChangeSupport vetoableChanges = new VetoableChangeSupport(this);

    private ArrayList<QrHistoryEntry> entries = new ArrayList<>();
    private int maxEntries = 50;
    private int discardedEntryCount = 0;

    public QrHistoryBean() {
    }

    public QrHistoryBean(int maxEntries) {
        this.maxEntries = maxEntries;
    }

    public String getComponentId() {
        return COMPONENT_ID;
    }

    // ------------------------------------------------------------------ indexed property

    /** @return a copy of the whole indexed property. */
    public QrHistoryEntry[] getEntries() {
        return entries.toArray(new QrHistoryEntry[0]);
    }

    /** Replaces the whole indexed property. */
    public void setEntries(QrHistoryEntry[] newEntries) {
        ArrayList<QrHistoryEntry> replacement = new ArrayList<>();
        if (newEntries != null) {
            Collections.addAll(replacement, newEntries);
        }
        trim(replacement);
        QrHistoryEntry[] old = getEntries();
        this.entries = replacement;
        propertyChanges.firePropertyChange("entries", old, getEntries());
    }

    /**
     * @param index position of the entry
     * @return the entry stored at that position
     */
    public QrHistoryEntry getEntries(int index) {
        return entries.get(index);
    }

    /** Replaces one slot of the indexed property and fires an indexed property change event. */
    public void setEntries(int index, QrHistoryEntry entry) {
```

`QrHistoryBean` stores recent generation records. Its maximum size can be changed, and older entries are removed when the limit is reached.

3.8 Desktop window

Desktop application window

```java
package ke.ku.sco310.qr.ui;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrGenerationResult;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.editor.HexColorPropertyEditor;
import ke.ku.sco310.qr.facade.QrAssembly;
import ke.ku.sco310.qr.render.AsciiRendererBean;
import ke.ku.sco310.qr.render.PngRendererBean;
import ke.ku.sco310.qr.render.SvgRendererBean;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Dimension;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Image;
import java.awt.Insets;
import java.awt.RenderingHints;
import java.awt.image.BufferedImage;
import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Locale;
import javax.swing.BorderFactory;
import javax.swing.DefaultComboBoxModel;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSplitPane;
import javax.swing.JTextArea;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.WindowConstants;

/** Swing assembly of the component suite (front end number one). */
public class QrStudioFrame extends JFrame implements PropertyChangeListener {

    private static final long serialVersionUID = 1L;

    private final transient QrAssembly assembly;

    private final JTextArea inputArea = new JTextArea(4, 34);
    private static final int DEFAULT_MODULE_SIZE = 10;
    private static final String DEFAULT_FOREGROUND = "#101820";
    private static final String DEFAULT_BACKGROUND = "#FFFFFF";

    private final JComboBox<String> rendererCombo = new JComboBox<>(
            new DefaultComboBoxModel<>(new String[]{"PNG", "SVG", "ASCII"}));
    private final FitLabel previewLabel = new FitLabel();
    private final JTextArea previewText = new JTextArea(24, 60);
    private final CardLayout previewCards = new CardLayout();
    private final JPanel previewDeck = new JPanel(previewCards);
    private final JTextArea historyArea = new JTextArea(14, 60);
```

`QrStudioFrame` reads the user input, calls the facade and displays the returned result. The QR encoding algorithm is kept outside the window class.

3.9 ZXing adapter

ZXing encoder adapter

```
package ke.ku.sco310.qr.components;

import ke.ku.sco310.qr.api.BoundPropertySource;
import com.google.zxing.EncodeHintType;
import com.google.zxing.WriterException;
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
import com.google.zxing.qrcode.decoder.Mode;
import com.google.zxing.qrcode.encoder.ByteMatrix;
import com.google.zxing.qrcode.encoder.Encoder;
import com.google.zxing.qrcode.encoder.QRCode;

import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.engine.QrTables;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.Serializable;
import java.util.Arrays;
import java.util.EnumMap;
import java.util.Map;

/** Adapter component that exposes the third-party ZXing engine through this suite's own QrCodeEncoder interface. */
public class ZxingQrEncoderAdapterBean implements QrCodeEncoder, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "encoder.zxing-adapter.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);

    private String characterSet = "UTF-8";
    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean boostErrorCorrection = false;

    public ZxingQrEncoderAdapterBean() {
    }

    @Override
    public QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException {
        QrErrorCorrection requested = level == null ? defaultErrorCorrection : level;
        ErrorCorrectionLevel zxingLevel = mapLevel(requested);
        try {
            if (boostErrorCorrection) {
                zxingLevel = ErrorCorrectionLevel.H;
            }
            Map<EncodeHintType, Object> hints = new EnumMap<>(EncodeHintType.class);
            hints.put(EncodeHintType.CHARACTER_SET, characterSet);
            QRCode encoded = Encoder.encode(content, zxingLevel, hints);
            return convert(encoded, content, requested);
        } catch (WriterException e) {
            throw new QrEncodingException(COMPONENT_ID,
                    "The ZXing engine refused the payload: " + e.getMessage(), e);
        } catch (RuntimeException e) {
            throw new QrEncodingException(COMPONENT_ID,
```

`ZxingQrEncoderAdapterBean` allows ZXing to be used through the same `QrCodeEncoder` interface. The rest of the application does not need to know which encoder is connected.

---

4. Components used

Role	Component	Main work
Encoder	`PureJavaQrEncoderBean`	creates standard QR Codes
Encoder	`ZxingQrEncoderAdapterBean`	uses the optional ZXing encoder
Micro QR encoder	`MicroQrEncoderBean`	creates Micro QR M1–M4 symbols
Decoder	`PureJavaQrDecoderBean`	reads supported QR matrices and generated files
Decoder	`ZxingQrDecoderAdapterBean`	uses ZXing for decoding
Renderer	`PngRendererBean`	creates PNG images
Renderer	`SvgRendererBean`	creates SVG files
Renderer	`AsciiRendererBean`	creates text output
Renderer	`RoundedModuleRendererBean`	creates rounded PNG images
Renderer	`PdfRendererBean`	creates PDF files
Repository	`FileSystemRepositoryBean`	saves files and stored history
History	`QrHistoryBean`	keeps recent generation records
Listener	`AuditLogListenerBean`	records generation events
Facade	`QrStudioFacadeBean`	controls the generation process

4.1 JavaBeans features used

The components use public no-argument constructors and getter/setter methods. Bound properties send change events when their values are updated. Constrained properties can reject invalid values. `QrHistoryBean` also provides indexed access to individual history entries.

`PngRendererBeanBeanInfo` and `QrHistoryBeanBeanInfo` provide extra information about the bean properties. `HexColorPropertyEditor` changes hexadecimal colour text into Java

`Color` objects.

---

5. QR Code generation

The built-in encoder follows these steps:

1. check the text and choose suitable encoding modes;
2. choose the smallest version that can hold the data;
3. add the mode, character count, data and padding bits;
4. create Reed–Solomon error-correction codewords;
5. place the finder, timing, alignment and data modules;
6. test the eight mask patterns; and
7. add the format and version information.

Numeric, alphanumeric, byte, Kanji, UTF-8 ECI and mixed-mode content are supported. Standard QR versions 1–40 and error-correction levels L, M, Q and H are available. Micro QR M1–M4 is handled by a separate encoder component.

---

6. Configuration and saved data

`config/assembly.xml` stores the selected components and their property values. `AssemblyProfile` loads this file, while `ComponentRegistry` creates and connects the components.

`FileSystemRepositoryBean` saves generated files in the configured folder. It also stores the history so that previous records can be loaded again. `AssemblyGraphBean` supports saving and restoring the connected bean objects through `XMLEncoder` and `XMLDecoder`.

---

7. Testing

The project includes checks for encoding, decoding, renderers, component properties, history, configuration and saved files. The tests also use ZXing to decode generated symbols and compare the result with the original text.

Encoder tests

```java
package ke.ku.sco310.qr.test;

import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.components.PureJavaQrEncoderBean;
import ke.ku.sco310.qr.engine.QrTables;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** Functional verification of the in-house encoder component against the independent ZXing oracle. */
final class EngineReferenceTest {

    private EngineReferenceTest() {
    }

    static void run(TestHarness harness, List<String> tableCrossCheckCsv, List<String> capacityCsv) {
        // ------------------------------------------------------------------ table cross-check
        int mismatches = 0;
        StringBuilder detail = new StringBuilder();
        for (int version = 1; version <= 40; version++) {
            for (QrErrorCorrection level : QrErrorCorrection.values()) {
                int[] zxing = ZxingVerify.zxingCodewordCounts(version, level);
                int eccPerBlock = QrTables.eccCodewordsPerBlock(level, version);
                int blockCount = QrTables.numErrorCorrectionBlocks(level, version);
                int dataCodewords = QrTables.numDataCodewords(version, level);
                int totalCodewords = QrTables.totalCodewords(version);
                boolean same = zxing[0] == eccPerBlock && zxing[1] == blockCount
                        && zxing[2] == dataCodewords && zxing[3] == totalCodewords;
                if (!same) {
                    mismatches++;
                    detail.append(String.format("v%d-%s ", version, level.getLabel()));
                }
                tableCrossCheckCsv.add(String.format("%d,%s,%d,%d,%d,%d,%d,%d,%d,%d,%s",
                        version, level.getLabel(), zxing[0], eccPerBlock, zxing[1], blockCount,
                        zxing[2], dataCodewords, zxing[3], totalCodewords,
                        same ? "match" : "MISMATCH"));
            }
        }
        harness.check("engine", "ISO 18004 tables match ZXing for 40 versions x 4 levels",
                mismatches == 0, mismatches + " mismatching combinations " + detail);

        // ------------------------------------------------------------------ payload round trips
        String[] payloads = {
            "A",
            "HELLO WORLD",
            "1234567890",
            "12345678901234567890123456789012345678901234567890",
            "Kenyatta University",
            "Kenyatta University - SCO310 Component Programming 2025/2026",
            "Mixed case, punctuation: .,;:!? and symbols /+-%*",
            "\u00e9\u00e8\u00ea caf\u00e9 na\u00efve r\u00e9sum\u00e9",
            "\u4f60\u597d \u4e16\u754c - Chinese text payload",
            "\ud83d\ude80 Emoji payload with a rocket",
            "tel:+254700000000",
            "WIFI:T:WPA;S:SCO310-Lab;P:component-programming;;",
            "BEGIN:VCARD\nVERSION:3.0\nFN:Component Programming\nEND:VCARD",
            "line1\nline2\tTAB\rcarriage",
            "0123456789abcdefghijklmnopqrstuvwxyz",
        };
        int decoded = 0;
        int failures = 0;
        for (String payload : payloads) {
            for (QrErrorCorrection level : QrErrorCorrection.values()) {
                try {
                    QrCode symbol = new PureJavaQrEncoderBean().encode(payload, level);
                    String back = ZxingVerify.decodeModules(symbol);
```

The latest full test run completed successfully. Invalid component names, missing connections and corrupted files are also tested to make sure errors are detected.

8. Problems fixed

8.1 Error-correction blocks

The first multi-block version placed some data and error-correction codewords in the wrong order. The interleaving step was corrected and checked with larger QR versions.

8.2 Large preview images

Large QR Codes could be cut off in the desktop window. The preview layout was changed so the result can fit or scroll correctly.

8.3 History property

The history entries were first shown as a normal property. `IndexedPropertyDescriptor` was added so individual entries can be discovered and accessed correctly.

8.4 Optional ZXing library

The application now checks whether ZXing is available before selecting its adapter. If it is not available, the built-in encoder remains selected.

9. Conclusion

QR Component Studio generates QR Codes through separate encoder, renderer, storage, history and listener components. These parts are connected through shared interfaces, which allows an encoder or renderer to be changed without changing the rest of the application.

The desktop application generates PNG, SVG and text output, saves the result and records recent operations. The built-in QR engine handles the complete generation process, while the optional ZXing adapter provides another encoder using the same interface.

10. References

1. International Organization for Standardization and IEC. *ISO/IEC 18004:2015 — QR Code bar code symbology specification*.
2. Szyperski, C. *Component Software: Beyond Object-Oriented Programming*. Second edition, Addison-Wesley, 2002.
3. Oracle. *JavaBeans Specification* and `java.beans` API documentation.
4. ZXing Project. *ZXing Core 3.5.3*.
5. SCO 310 / SIT 300 Component Programming course materials, Kenyatta University.